

Programação Concorrente - Primeira Lista de Exercícios

Instituto de Computação - Prof. Gabriel P. Silva

1. Em relação aos processos e threads, assinale as alternativas como verdadeiras ou falsas: (0,5 ponto)
 - () Em um sistema com threads, se uma thread alterar o valor de uma variável global, essa alteração é visível para todas as outras threads do mesmo processo.
 - () Criar um processo via fork é mais rápido do que criar uma thread POSIX, pela ausência de mecanismos de sincronização no momento da criação do processo.
 - () A comunicação entre processos exige o uso de mecanismos como pipes, soquetes ou troca de mensagens, devido ao seu isolamento de memória.
 - () Para servidores de rede a criação de um processo independente para cada conexão é sempre a estratégia mais eficiente do que o uso de threads.
 - () O uso de processos é mais apropriado em cenários onde a segurança e a robustez são prioridades, pois o isolamento impede que um erro em um fluxo de execução corrompa a memória de outro.
2. Para cada cenário abaixo, indique qual(is) API(s) seria(m) mais apropriada(s) e justifique: (0,5 ponto)
 - a. Paralelizar laços em um programa científico C++ que roda em um servidor com 64 núcleos
 - b. Acelerar multiplicação de matrizes usando GPU NVIDIA
 - c. Simular um sistema climático global em um supercomputador com 1000 nós
 - d. Processar imagens médicas em diferentes tipos de aceleradores (GPUs AMD, Intel, NVIDIA)
 - e. Desenvolver uma simulação de dinâmica molecular de altíssimo desempenho para um cluster onde cada nó possui 32 núcleos de CPU e duas GPUs.
3. Com relação aos ambientes e paradigmas de programação concorrente e paralela, responda: (1,0 ponto)
 - a. Por que muitas aplicações de alto desempenho combinam MPI com OpenMP ou CUDA? Que vantagens essa combinação oferece?
 - b. O que são condições de corrida (race conditions)? Por que elas são problemáticas em programas concorrentes? Dê um exemplo hipotético.
 - c. Qual o impacto que o desbalanceamento de carga tem no desempenho final das aplicações paralelas? (0,5 ponto)
 - d. Um sistema operacional moderno gerencia a concorrência em múltiplos níveis. Identifique e explique pelo menos três níveis onde a concorrência aparece em um SO.
 - e. Explique a diferença entre corrotinas (async/await) e threads tradicionais. Em que tipo de aplicação as corrotinas são particularmente vantajosas?

4. Escreva um pequeno programa em C que utilize `fork()` para criar três processos filhos. Cada filho deve imprimir seu PID e o PID do pai, enquanto o pai deve esperar por todos antes de encerrar. (1,0 ponto)
5. No exemplo da soma de um vetor, a variável **somatotal** é compartilhada entre threads. Descreva por que a operação **somatotal = somatotal + somalocal** precisa de exclusão mútua, e as atualizações **soma_local** e o índice **'i'** do laço interno da thread não. (0,5 ponto)
6. Em que situação específica um programador precisaria configurar um mutex como `PTHREAD_MUTEX_RECURSIVE`? O que acontece se uma thread tentar travar duas vezes um mutex comum (não recursivo)? (0,5 ponto)
7. Com relação ao código de contagem de primos, responda: (0,5 ponto)
 - a. Na versão com divisão em blocos, explique por que a distribuição estática de trabalho pode gerar desequilíbrio entre as threads.
 - b. Na versão com saco de tarefas, o índice global **i_global** é protegido por um mutex. Explique por que essa proteção é necessária e qual trecho do código constitui a seção crítica.
8. Por que a versão do cálculo de número primos com saco de tarefas apresenta melhor balanceamento de carga em comparação com a divisão em blocos? Por que a sua eficiência melhorou, mesmo usando mutex? (0,5 ponto)
9. Quais são as quatro formas principais de sincronização citadas para programas de memória compartilhada? Como são mapeadas em bibliotecas da linguagem 'C'? (0,5 ponto)
10. Com relação ao uso de sincronização por variáveis condicionais, responda: (0,6 ponto)
 - a. Qual a diferença funcional entre as operações `SIGNAL` e `BROADCAST` em uma variável de condição.
 - b. Por que uma variável de condição deve ser sempre usada em conjunto com um mutex?
 - c. O que acontece com o lock (mutex) quando uma thread executa a operação `pthread_cond_wait()` e o que ocorre quando ela retoma a execução?
11. Com relação à sincronização com uso de barreiras, responda: (0,4 ponto)
 - a. Defina o conceito de sincronização por barreira e explique por que ela é comparada a um "ponto de encontro". (0,5 ponto)
 - b. Quais são as rotinas da biblioteca Pthreads utilizadas para inicializar, esperar e destruir uma barreira? (0,5 ponto)
12. Com relação aos semáforos, responda: (1,0 ponto)
 - a. Qual a diferença entre semáforo binário e um mutex?

- b. Qual a diferença entre um semáforo de contagem e um semáforo binário?
 - c. Em que situações um semáforo é mais útil que um mutex?
 - d. Quais são as duas operações atômicas utilizadas para acessá-los?
 - e. Quais as principais limitações de um semáforo?
13. Explique a diferença de escopo entre um semáforo inicializado com **sem_init()** e um aberto com **sem_open()**. Qual deles é mais adequado para sincronizar dois processos independentes que não compartilham memória? (0,5 ponto)
14. Implementar uma rotina para emular o comportamento de uma barreira para 3 threads utilizando apenas um Mutex e uma Variável de Condição (sem usar `pthread_barrier_t`). A última thread a chegar deve dar o sinal para todas as outras. (1,0 ponto)
15. Estenda o exemplo de produtor-consumidor para usar um array (buffer) de 5 posições. O produtor só pode produzir se houver espaço, e o consumidor só pode consumir se o buffer não estiver vazio. Utilize variáveis de condição para gerenciar esses estados. (1,0 ponto)